

7-3 · Embedding과 Vector DB · 실습 1.5H

Qdrant 컬렉션 생성·적재·검색 실습

띄우고, 만들고, 넣고, 찾고, 걸러 본다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① Qdrant 를 볼륨과 함께 띄우고 대시보드를 연다
- ② size·distance 를 지정해 컬렉션을 만들고, 차원을 일부러 틀려 본다
- ③ payload 를 담아 적재하고 query_points 로 의미 검색한다
- ④ payload 필터를 걸고, 7-2의 조각 전체를 적재·검색한다

8단계의 검색 결과가 7-5에서 LLM 에게 넘길 「근거」가 됩니다

여덟 단계 — 컨테이너 하나, 컬렉션 둘

1H 실습에서 만든 emb 객체와 v1 을 그대로 이어서 씁니다

1. 기동	2. 컬렉션	3. 실패	4·5·6. 적재·검색	7·8. 조각
docker run curl 확인	size=DIM COSINE	size=10 일부러 틀리기	upsert query_points·필터	7-2 조각 적재 근거 확인

1단계

Qdrant 를 띄우고 응답을 확인

기동	<code>docker run -d --name qdrant ... qdrant/qdrant</code>
포트	<code>-p 6333:6333 -p 6334:6334</code>
볼륨	<code>-v qdrant_storage:/qdrant/storage</code>
확인	<code>docker ps --filter name=qdrant</code>
응답	<code>curl http://localhost:6333/collections</code>
대시보드	<code>http://localhost:6333/dashboard</code>

`{"result":{"collections":[]}}...` 이 오면 정상 · 볼륨이 없으면 컨테이너를 지울 때 적재한 벡터가 전부 사라진다

size 와 distance 를 지정해 컬렉션 생성

1

QdrantClient(url=...)

http://localhost:6333 — 대시보드와 같은 포트

2

DIM = len(v1)

임베딩 모델의 차원과 반드시 일치 — 1H 실습에서 메모한 값

3

없을 때만 create_collection

collection_exists() 확인 → VectorParams(size=DIM, distance=COSINE)

출력에서 볼 것

size

distance

컬렉션 이름 lecture_notes — 대시보드의 Collections 목록에도 나타난다

차원을 틀린 컬렉션에 넣어 본다

Vector dimension error 계열 메시지 — 모델을 바꿨을 때 만나는 바로 그 오류를 미리 봐 둡니다

wrong_dim 생성	v1 을 upsert	except 로 잡기	삭제
size=10 COSINE	1536(768)차원 벡터를 넣기	type(e).__name__ str(e)[:300]	delete_collection ("wrong_dim")

payload 를 담아 여섯 문장 적재

id	text	category
0	ETL은 추출, 변환, 적재 순서로...	개념
1	ELT는 먼저 적재한 뒤 안에서 변환	개념
2	Airflow는 워크플로 오케스트레이션...	도구
3	Kafka는 이벤트 스트리밍 플랫폼	도구
4	Iceberg는 오브젝트 스토리지 위의...	저장
5	Spark는 대용량 데이터를 분산 처리	처리

payload = {text, category, source: "lecture"} — 대시보드에서 포인트를 눌러 payload 가 함께 들어갔는지 본다

emb.embed_documents(texts) 로 배치 임베딩 → PointStruct(id, vector, payload) → client.upsert · count=6 이면 성공

의미 검색, 그리고 payload 필터

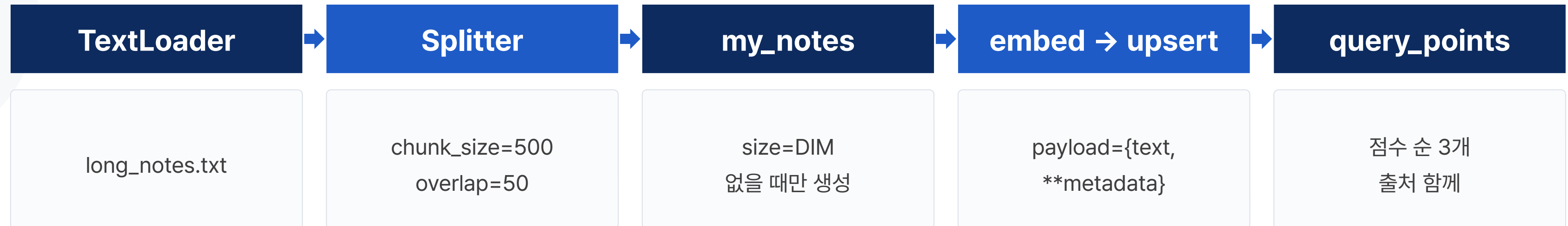
- 1 search(q) — client.query_points**
query=emb.embed_query(q), limit=3, query_filter=flt → .points
- 2 「작업 순서를 관리하는 도구는?」**
Airflow 문장이 1등 — 질문에 Airflow 도 오케스트레이션도 없다
- 3 Filter(must=[FieldCondition(...)])**
key="category", match=MatchValue(value="도구") — 도구만 남는다

실무에서는

**이 부서만
올해 문서만**

의미 검색과 조건 검색을 함께 쓰는 방식 — 「데이터를 옮기는 방법」을 필터 없이·있이 두 번 비교

7-2에서 만든 조각을 통째로 넣고 검색



조각 수와 count 가 같으면 성공 — 7-2에서 정한 chunk_size 를 그대로 쓴다 · 이것이 7-5에서 LLM 에게 넘길 「근거」

「배치와 스트리밍은 어떻게 다른가요?」— 관련 조각이 점수 순으로 나오고 출처(source)가 함께 찍힌다

이 1.5시간이 끝날 때 확인할 것

- 1 `curl .../collections` 가 JSON 을 돌려주고 대시보드가 열린다
- 2 `lecture_notes` 의 `size·distance` 가 출력됐다
- 3 `wrong_dim` 에 넣으니 `Vector dimension error` 가 났다
- 4 `client.count` 가 6 · 대시보드에서 `payload` 가 보인다
- 5 `Airflow` 없는 질문에 `Airflow` 문장이 1등이다
- 6 필터를 걸면 `category="도구"` 만 나온다
- 7 `my_notes` 의 `count` 가 조각 수와 같고 출처가 찍힌다

7-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

오늘 쓴 명령을 표로 정리하고, 자주 겪는 오류 네 가지를 증상으로 찾아 보고, 7-4 MinIO 연동을 준비합니다

✓ 임베딩·Qdrant 명령 요약표

✓ 오늘의 체크리스트 14개

✓ 오류: dimension·refused·409·엉뚱한 결과

✓ 7-4 준비: qdrant·minio 둘 다 Up

가져갈 것

더 있는 qdrant 컨테이너 그대로